

OCTA Research

Emergence in Modern Computation

Decision Logic Trees → Coarse-Grained Macro Laws → Context/Prompt Regimes → OS-Layer Evolution

Rigorous Formalization with Operational Metrics, Phase Regimes, and Stack-Level Interventions

Version: v1.0 (2026-02-03)

Abstract

We formalize *emergence* in modern-day computing as the appearance of stable, compressible, and approximately autonomous macro-dynamics induced by local decision logic and state transitions. The framework unifies: (i) deterministic programs and decision trees as piecewise dynamical systems; (ii) interacting multi-component computation as coupled stochastic processes; (iii) philosophical prompting and conversation context as constraint-driven interventions that create macro-level attractors in context space; and (iv) naturalized software evolution and OS upgrades as layer interventions that reconfigure effective transition kernels and thus the emergent macro laws (e.g., tail latency regimes, nondeterminism, failure clusters). We provide definitions (lumpability, approximate closure), information-theoretic metrics (conditional mutual information, transfer entropy), MDL-based compressibility criteria, and a practical discovery objective for extracting macro variables where emergence “lives.” TikZ diagrams encode the full stack and coarse-graining flow.

Key claim. Emergence is neither mystical nor uniquely biological: it is a *coarse-grained closure phenomenon* arising from *composition, interaction, thresholds, and feedback*, producing macro variables that are (a) predictive and (b) simpler to model than micro dynamics.

Contents

1	System Model: Microstate, Transition, and Observation	2
1.1	Microstate and Dynamics	2
1.2	Macro-Observation via Coarse-Graining	2
2	Decision Logic Trees and Programs as Piecewise Dynamical Systems	2
2.1	Branching Predicates and Piecewise Maps	2
2.2	Feedback Turns Static Trees into Dynamics	2
3	Interacting Computation: Coupling, Thresholds, and Scale	2
3.1	Networked Microdynamics	2
3.2	Threshold Logic (Canonical Nonlinearity)	3
4	Emergence: Formal Definitions (Autonomy + Compressibility)	3
4.1	Exact Macro Closure: Lumpability	3
4.2	Approximate Closure: Quantitative Emergence	3
4.3	Information-Theoretic Autonomy (Estimable)	3
4.4	Compressibility: Macro Laws Simpler than Micro Simulation	4
4.5	Unified Emergence Definition	4

5	Directed Influence and Synergy Metrics (Rigorous Statistics)	4
5.1	Transfer Entropy	4
5.2	Synergy (When the “Whole” Carries More Information)	4
5.3	Causal-State Emergence (Computational Mechanics)	4
6	Emergence from Philosophical Prompting and Context	5
6.1	Context as State: The Prompted System	5
6.2	Prompting as an Intervention Operator	5
6.3	Constraint-Tilted Inference (Philosophy as Lagrange Multipliers)	5
6.4	Emergent “Frames” as Macro Variables in Context Space	5
7	Emergence from Naturalized Origin Code and OS Upgrades	6
7.1	Layered Stack as Composed Transition Kernels	6
7.2	OS Upgrade as Kernel Intervention	6
7.3	Concurrency as Mixture over Interleavings	6
7.4	Naturalized Software Evolution as Path-Dependent Kernel Process	6
8	Renormalization View: Coarse-Graining Across Layers	6
8.1	Induced Macro Kernel	6
8.2	Regime Shift (Phase Change) Criterion	7
9	Discovery: Finding the Macro Variables Where Emergence Lives	7
9.1	Predictive Closure via Comparative Modeling	7
9.2	A Principled Search Objective	7
10	TikZ Diagrams (OCTA Stack + Coarse-Graining + Interventions)	7
10.1	Full Stack with Interventions	8
10.2	Coarse-Grained Closure (Macro Autonomy)	8
11	Operational Study Design (Full Scope)	8
11.1	Domains Covered	8
11.2	Canonical Macro Observables	9
11.3	Estimation and Testing Protocol	9
12	Selected Theorems (Anchor Results)	9
13	Glossary (OCTA Terms for This Study)	9
14	Conclusion (Full-Scope Summary)	10

1 System Model: Microstate, Transition, and Observation

1.1 Microstate and Dynamics

Definition 1.1 (Microstate space and dynamics). Let $\mathcal{X}$ be a (finite or measurable) microstate space and let $(X_t)_{t \geq 0}$ be a stochastic process on $\mathcal{X}$. The most general *one-step* dynamics are specified by a Markov kernel P :

$$\mathbb{P}(X_{t+1} \in B \mid X_t = x) = P(B \mid x), \quad \forall x \in \mathcal{X}, B \subseteq \mathcal{X}.$$

In density form, $p(x' \mid x)$ denotes the conditional density.

Definition 1.2 (Deterministic computation as a special case). A deterministic program corresponds to a measurable function $F : \mathcal{X} \rightarrow \mathcal{X}$ such that

$$X_{t+1} = F(X_t) \iff p(x' \mid x) = \delta(x' - F(x)).$$

1.2 Macro-Observation via Coarse-Graining

Definition 1.3 (Coarse-graining / macro map). A macro-observable (macrostate) is induced by a map $g : \mathcal{X} \rightarrow \mathcal{Y}$ where $\mathcal{Y}$ is typically lower-dimensional. Define $Y_t := g(X_t)$.

Remark 1.1. In practice, g may represent: averages, cluster labels, graph connectivity statistics, entropy/complexity indices, regime labels (normal vs saturated), or conversational “frame” variables (stance, constraints, rigor).

2 Decision Logic Trees and Programs as Piecewise Dynamical Systems

2.1 Branching Predicates and Piecewise Maps

Definition 2.1 (Piecewise dynamical computation). Let $\{\phi_\ell\}_{\ell=1}^L$ be predicates $\phi_\ell : \mathcal{X} \rightarrow \{0, 1\}$ defining a partition into regions $R_\ell = \{x : \phi_\ell(x) = 1\}$ (assume exactly one region active). A program with branching is the map

$$F(x) = F_\ell(x) \quad \text{if } x \in R_\ell.$$

Remark 2.1. Decision trees are canonical examples: the predicates are comparisons $z_j \leq \tau$, and each leaf applies a constant action/label.

2.2 Feedback Turns Static Trees into Dynamics

Definition 2.2 (Closed-loop decision tree controller). Let $S_t \in \mathcal{S}$ be a system state, $A_t \in \mathcal{A}$ an action, and $P(\cdot \mid s, a)$ a controlled transition kernel. A decision tree policy $f : \mathcal{S} \rightarrow \mathcal{A}$ yields closed-loop dynamics:

$$A_t = f(S_t), \quad S_{t+1} \sim P(\cdot \mid S_t, f(S_t)).$$

Remark 2.2. Even if f is simple, the induced process on S_t can exhibit attractors, cycles, and regime switching because f partitions state space and selects distinct local transition kernels.

3 Interacting Computation: Coupling, Thresholds, and Scale

3.1 Networked Microdynamics

Let $X_t = (X_t^{(1)}, \dots, X_t^{(N)}) \in \mathcal{X}^N$, with interaction graph G and neighborhoods $\mathcal{N}(i)$. A broad coupled class is:

$$X_{t+1}^{(i)} \sim P_i(\cdot \mid X_t^{(i)}, X_t^{(\mathcal{N}(i))}).$$

3.2 Threshold Logic (Canonical Nonlinearity)

Definition 3.1 (Threshold-coupled update). A deterministic threshold update is

$$X_{t+1}^{(i)} = \mathbf{1} \left(\sum_{j \in \mathcal{N}(i)} w_{ij} X_t^{(j)} - \theta_i > 0 \right).$$

Remark 3.1. Thresholding is a ubiquitous computational primitive: if-statements, admission control, rate limiting, branch prediction boundaries, scheduling heuristics, congestion control triggers. Thresholds + coupling + iteration are sufficient for phase transitions and emergent macro laws.

4 Emergence: Formal Definitions (Autonomy + Compressibility)

4.1 Exact Macro Closure: Lumpability

Definition 4.1 (Strong lumpability). Let $Y_t = g(X_t)$. The process is *strongly lumpable* w.r.t. g if for all $y \in \mathcal{Y}$ and all $x_1, x_2 \in g^{-1}(y)$,

$$\mathbb{P}(Y_{t+1} = y' \mid X_t = x_1) = \mathbb{P}(Y_{t+1} = y' \mid X_t = x_2) \quad \forall y' \in \mathcal{Y}.$$

Equivalently, there exists a macro kernel Q such that

$$\mathbb{P}(Y_{t+1} = y' \mid Y_t = y) = Q(y' \mid y)$$

and the value is independent of microstate within each macro class.

4.2 Approximate Closure: Quantitative Emergence

Definition 4.2 (Approximate closure error). Define the induced macro kernel

$$Q(y' \mid y) = \mathbb{P}(g(X_{t+1}) = y' \mid g(X_t) = y).$$

The closure error at macrostate y may be quantified by total variation:

$$\epsilon_{\text{TV}}(y) = \sup_{x \in g^{-1}(y)} D_{\text{TV}} \left(\mathbb{P}(Y_{t+1} \in \cdot \mid X_t = x), Q(\cdot \mid y) \right),$$

or KL divergence:

$$\epsilon_{\text{KL}}(y) = \sup_{x \in g^{-1}(y)} D_{\text{KL}} \left(\mathbb{P}(Y_{t+1} \mid X_t = x) \parallel Q(\cdot \mid y) \right).$$

4.3 Information-Theoretic Autonomy (Estimable)

Definition 4.3 (Macro autonomy via conditional mutual information). Define

$$\Delta := I(Y_{t+1}; X_t \mid Y_t).$$

If $\Delta = 0$, then given Y_t , the microstate X_t adds no predictive information about Y_{t+1} . Small Δ indicates approximate macro closure.

4.4 Compressibility: Macro Laws Simpler than Micro Simulation

Definition 4.4 (MDL emergence). Let $\mathcal{Q}$ be a restricted class of macro models (e.g., small Markov models, GLMs, low-order state-space models). For $Q \in \mathcal{Q}$, define

$$\text{MDL}(Q; y_{1:T}) = L(Q) + L(y_{1:T} | Q),$$

where $L(Q)$ encodes model complexity and $L(y_{1:T} | Q)$ is negative log-likelihood (or residual code length). A macro description is *emergent* if there exists $Q^* \in \mathcal{Q}$ with low MDL while maintaining strong predictive performance.

4.5 Unified Emergence Definition

Definition 4.5 (Emergence under coarse-graining). Fix $g : \mathcal{X} \rightarrow \mathcal{Y}$ and induced $Y_t = g(X_t)$. We say *emergent macro-dynamics* occur if:

- (i) **Autonomy:** $I(Y_{t+1}; X_t | Y_t) \leq \delta$ for small δ .
- (ii) **Nontrivial structure:** $I(Y_{t+1}; Y_t) \geq \alpha > 0$.
- (iii) **Compressibility:** there exists a low-complexity $Q \in \mathcal{Q}$ with small $\text{MDL}(Q; y_{1:T})$ and good held-out predictive loss.

5 Directed Influence and Synergy Metrics (Rigorous Statistics)

5.1 Transfer Entropy

Definition 5.1 (Transfer entropy). For component groups A and B with histories, transfer entropy is

$$\text{TE}_{A \rightarrow B} := I(X_{t+1}^{(B)}; X_t^{(A)} | X_t^{(B)}).$$

It measures directed predictive influence beyond self-history.

5.2 Synergy (When the “Whole” Carries More Information)

Definition 5.2 (Synergy diagnostic (simple form)). For predictors X_1, X_2 and target Z ,

$$\text{Syn}(Z; X_1, X_2) := I(Z; X_1, X_2) - I(Z; X_1) - I(Z; X_2).$$

Positive values indicate that predictive information is primarily present in the joint state, not in parts alone.

5.3 Causal-State Emergence (Computational Mechanics)

Definition 5.3 (Causal states and statistical complexity). Let pasts $x_{-\infty:t}$ be equivalent if they induce the same future distribution. Define the causal state mapping

$$\epsilon(x_{-\infty:t}) := \left\{ x'_{-\infty:t} : \mathbb{P}(X_{t+1:\infty} | x_{-\infty:t}) = \mathbb{P}(X_{t+1:\infty} | x'_{-\infty:t}) \right\}.$$

The statistical complexity is

$$C_\mu := H(\epsilon).$$

Increases in C_μ correspond to the emergence of structured predictive memory.

6 Emergence from Philosophical Prompting and Context

6.1 Context as State: The Prompted System

Definition 6.1 (Context-augmented microstate). Let $X_t = (S_t, C_t)$ where S_t is system state and C_t is context state (conversation history, constraints, config, environment). Dynamics:

$$(S_{t+1}, C_{t+1}) \sim P(\cdot \mid S_t, C_t).$$

6.2 Prompting as an Intervention Operator

Definition 6.2 (Prompt regime intervention). A prompting regime π acts as an intervention operator on context:

$$C_t^+ = I_\pi(C_t),$$

followed by state evolution:

$$(S_{t+1}, C_{t+1}) \sim P(\cdot \mid S_t, C_t^+).$$

6.3 Constraint-Tilted Inference (Philosophy as Lagrange Multipliers)

Definition 6.3 (Constraint-tilted next-output distribution). Let $p_\theta(a \mid C)$ be a base next-output distribution over $a \in \mathcal{A}$. A philosophical constraint set $\mathcal{K}_\pi$ induces a constrained projection:

$$q_\pi(\cdot \mid C) = \arg \min_q D_{\text{KL}}(q(\cdot) \parallel p_\theta(\cdot \mid C)) \quad \text{s.t.} \quad \mathbb{E}_q[\phi_k(a, C)] \geq b_k \quad \forall k.$$

Under standard regularity, the solution has exponential-tilt form:

$$q_\pi(a \mid C) \propto p_\theta(a \mid C) \exp\left(\sum_k \lambda_k \phi_k(a, C)\right).$$

Remark 6.1. The multipliers λ_k encode the active philosophical/epistemic constraints (rigor, moral lens, ontology, safety, etc.). Prompting changes the effective dynamics by changing these constraint weights and/or the constraint functions.

6.4 Emergent “Frames” as Macro Variables in Context Space

Define macro frame variables:

$$Y_t := g(S_t, C_t)$$

where Y_t may include stance/ontology labels, constraint-weight vectors, rigor level, objective proxy, or discourse regime. Emergence in prompting occurs when:

$$I(Y_{t+1}; (S_t, C_t) \mid Y_t) \text{ is small,}$$

meaning the frame is a sufficient macrostate for predicting its own evolution.

Proposition 6.1 (Attractor formation in context space (informal)). *If the context update rule $C_{t+1} = \text{append}(C_t, T_{t+1})$ and the generator $T_{t+1} \sim q_\pi(\cdot \mid C_t)$ tends to reinforce the active constraints, then the induced macro frame Y_t admits stable fixed points or slow manifolds (attractor-like regimes) under the induced macro map $Y_{t+1} \approx \Phi(Y_t)$.*

7 Emergence from Naturalized Origin Code and OS Upgrades

7.1 Layered Stack as Composed Transition Kernels

Definition 7.1 (Layer-composed execution kernel). Let layers be hardware/firmware H , OS/kernel K , runtime/libraries R , and application A . The effective one-step kernel may be represented conceptually as a composition:

$$P = P_A \circ P_R \circ P_K \circ P_H,$$

or in deterministic form:

$$F = F_A \circ F_R \circ F_K \circ F_H.$$

7.2 OS Upgrade as Kernel Intervention

Definition 7.2 (OS upgrade intervention). An OS upgrade changes $P_K \rightarrow P'_K$, inducing a new global kernel

$$P' = P_A \circ P_R \circ P'_K \circ P_H.$$

Even with identical application code, macro observables $Y = g(\text{execution traces})$ may change phase due to altered scheduling, memory management, security mitigations, etc.

7.3 Concurrency as Mixture over Interleavings

Definition 7.3 (Scheduler-induced mixture). Let Σ denote possible schedules/interleavings and $P_\sigma(\cdot | x)$ the transition under schedule σ . Then:

$$\mathbb{P}(X_{t+1} \in B | X_t = x) = \sum_{\sigma \in \Sigma} \pi(\sigma | x) P_\sigma(B | x).$$

An OS update can change π , yielding new macro regimes (tail latency, deadlock probability, throughput collapse thresholds).

Remark 7.1. This is a direct, formal explanation for “emergent bugs” and “new performance cliffs” after OS updates: the distribution over micro-trajectories changes even if application logic is unchanged.

7.4 Naturalized Software Evolution as Path-Dependent Kernel Process

Definition 7.4 (Kernel evolution process). Let P_t denote the effective kernel at version t . An engineering update process can be modeled as

$$P_{t+1} = \mathcal{U}(P_t, \Delta_t),$$

where Δ_t is a patch (feature, mitigation, optimization), and $\mathcal{U}$ encodes constraints (backward compatibility, ABI stability, regression tests, incentives). This yields path dependence: P_t depends on the full update history, not only on current specs.

8 Renormalization View: Coarse-Graining Across Layers

8.1 Induced Macro Kernel

Definition 8.1 (Induced macro transition kernel). Given $g : \mathcal{X} \rightarrow \mathcal{Y}$, define:

$$Q(y' | y) := \mathbb{P}(g(X_{t+1}) = y' | g(X_t) = y).$$

Changing prompting constraints or OS layers changes the micro kernel P , which changes Q . Emergent “phases” correspond to qualitative changes in Q (new attractors, bifurcations, heavy tails, failure clusters).

8.2 Regime Shift (Phase Change) Criterion

Definition 8.2 (Regime shift in macro dynamics). Let η be an intervention parameter (prompt regime or OS version). A regime shift occurs when

$$Q_\eta(\cdot \mid y) \not\approx Q_{\eta'}(\cdot \mid y)$$

in a way that yields qualitatively distinct macro behavior (e.g., new stable states, oscillations, tail risk).

9 Discovery: Finding the Macro Variables Where Emergence Lives

9.1 Predictive Closure via Comparative Modeling

Let $\{g_k\}$ be candidate coarse-grainings. For each k , fit:

$$\hat{q}_k(y_{t+1} \mid y_t) \quad \text{and} \quad \hat{r}_k(y_{t+1} \mid y_t, \phi(x_t))$$

and compare held-out log-loss:

$$\Delta\mathcal{L}_k := \mathcal{L}(\hat{q}_k) - \mathcal{L}(\hat{r}_k).$$

Small $\Delta\mathcal{L}_k$ indicates closure; $I(Y_{t+1}; Y_t)$ indicates nontrivial structure.

9.2 A Principled Search Objective

Definition 9.1 (Emergence score objective). Define

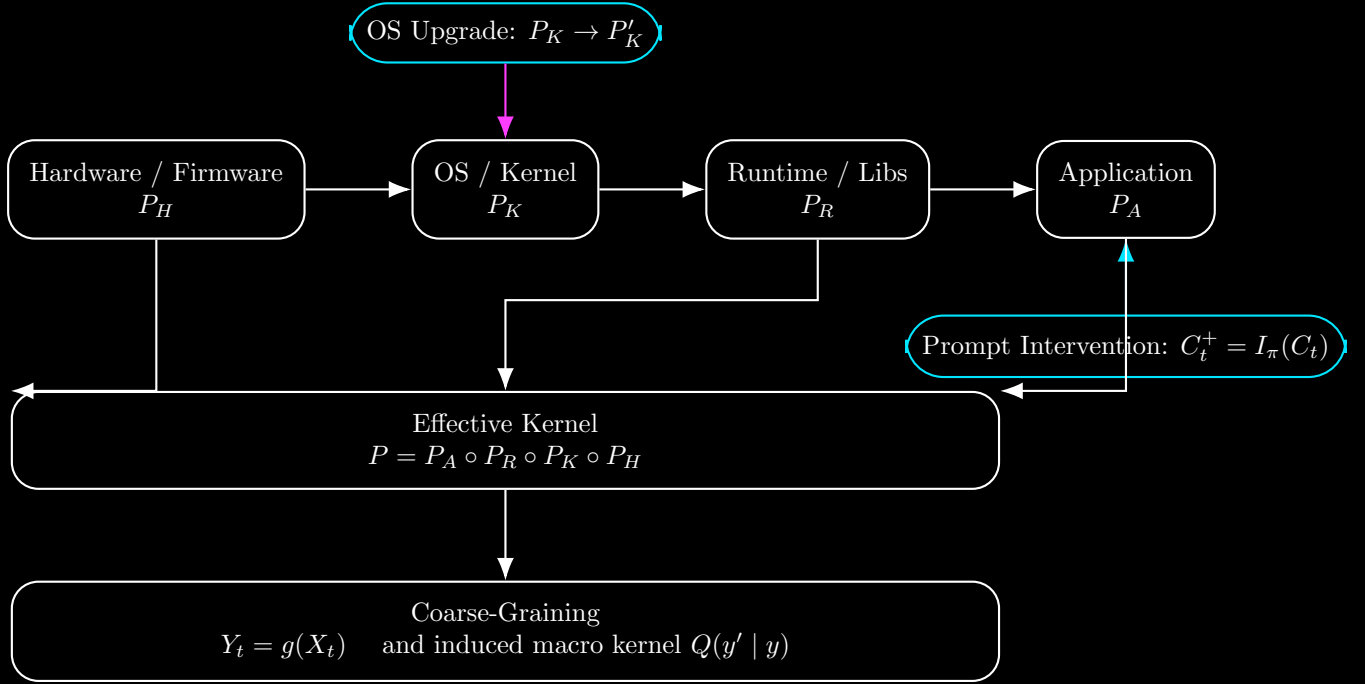
$$\text{Score}(g) := \underbrace{\hat{I}(Y_{t+1}; Y_t)}_{\text{structure}} - \lambda \underbrace{\hat{I}(Y_{t+1}; X_t \mid Y_t)}_{\text{closure violation}} - \gamma \underbrace{\text{Complexity}(g)}_{\text{overfit penalty}},$$

where hats denote estimators from logs.

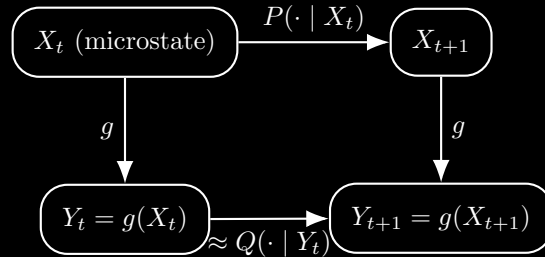
Remark 9.1. This gives a concrete optimization lens: maximize macro predictability while minimizing dependence on micro details and penalizing overly complex macro maps.

10 TikZ Diagrams (OCTA Stack + Coarse-Graining + Interventions)

10.1 Full Stack with Interventions



10.2 Coarse-Grained Closure (Macro Autonomy)



Macro autonomy criterion:

$\Delta = I(Y_{t+1}; X_t | Y_t)$ small $\Rightarrow$ micro adds little beyond macro for predicting next macro.
 Exact case: lumpability (well-defined Q independent of microstate within macro class).

11 Operational Study Design (Full Scope)

11.1 Domains Covered

We explicitly scope emergence in:

1. **Decision logic trees** (static rules) under **feedback** (closed-loop control).
2. **Programs** as piecewise maps with branch predicates.
3. **Multi-component systems** (distributed stacks, schedulers, networks, caches) where coupling induces phase regimes.
4. **Philosophical prompting and conversation context** as constraint-driven interventions that create stable macro frames.

5. **Naturalized origin code and OS upgrades** as layer-level interventions that alter effective kernels and thus macro laws.

11.2 Canonical Macro Observables

Candidate $Y_t = g(X_t)$ examples:

- **Performance phase:** normal / saturated / collapse; tail-latency regime labels (p95/p99 bins).
- **Failure clusters:** crash signatures, exception regimes, deadlock states.
- **Nondeterminism index:** variance across repeated identical runs.
- **Network equilibrium:** congestion window regimes, queue occupancy classes.
- **Prompt frame:** stance/ontology label, rigor level, constraint-weight vector λ , objective proxy.

11.3 Estimation and Testing Protocol

1. **Choose** candidate coarse-grainings g_k (hand-designed + learned).
2. **Fit** macro predictors $\hat{q}_k(y_{t+1} | y_t)$ and enriched predictors $\hat{r}_k(y_{t+1} | y_t, \phi(x_t))$.
3. **Measure closure:** $\Delta\mathcal{L}_k$ and/or $\hat{I}(Y_{t+1}; X_t | Y_t)$.
4. **Measure structure:** $\hat{I}(Y_{t+1}; Y_t)$, $\widehat{\text{TE}}$, synergy.
5. **Assess compressibility:** MDL($\hat{q}_k$) and generalization.
6. **Scan interventions:** prompting regimes π and OS/kernel versions u , detect regime shifts via changes in Q .

12 Selected Theorems (Anchor Results)

Theorem 12.1 (Lumpability implies Markov macro process). *If (X_t) is a Markov chain and is strongly lumpable w.r.t. g , then $Y_t = g(X_t)$ is Markov with transition kernel $Q(y' | y)$.*

Proof. By lumpability, for any $x \in g^{-1}(y)$, the distribution of Y_{t+1} depends only on y , not on x . Thus $\mathbb{P}(Y_{t+1} = y' | Y_t = y, Y_{t-1}, \dots) = \mathbb{P}(Y_{t+1} = y' | Y_t = y) = Q(y' | y)$. $\square$

Proposition 12.1 (Conditional mutual information detects closure). *If $I(Y_{t+1}; X_t | Y_t) = 0$, then $\mathbb{P}(Y_{t+1} | X_t) = \mathbb{P}(Y_{t+1} | Y_t)$ almost surely; i.e., macro is sufficient for next-macro prediction.*

Proof. $I(Y_{t+1}; X_t | Y_t) = 0$ iff $Y_{t+1} \perp X_t | Y_t$, which implies equality of the corresponding conditional distributions. $\square$

13 Glossary (OCTA Terms for This Study)

Microstate X_t

Full detailed computational state (system + context), including hidden stack effects.

Coarse-graining g

A mapping from microstate to macrostate, extracting a predictive summary.

Macro autonomy

The degree to which Y_t predicts Y_{t+1} without needing micro details.

Lumpability

Exact condition that makes the macro process Markov under g .

MDL emergence

A macro law that is both predictive and significantly simpler than micro simulation.

Prompt regime π

A constraint/intervention shaping effective inference, expressible as exponential tilting.

OS upgrade intervention

A change to kernel/runtime kernels (e.g., scheduler, allocator) altering effective system dynamics.

14 Conclusion (Full-Scope Summary)

We have formalized emergence across modern computation end-to-end: from deterministic logic and decision trees to networked coupled dynamics, from prompting-as-intervention to OS upgrades as kernel-level interventions. The unifying structure is the induced macro kernel Q under coarse-graining and the operational emergence criteria: (i) autonomy $I(Y_{t+1}; X_t | Y_t)$ small, (ii) nontrivial structure $I(Y_{t+1}; Y_t)$ large, and (iii) compressibility via MDL. This provides a rigorous, testable program for OCTA Research: measure, discover, and engineer emergent macro laws in both conversational context systems and real software stacks.

OCTA Research — Noir Formal Systems Series

“The Environment becomes your weapon.”